

DATA ARCHITECTURE

Standards and Guidelines

TABLE OF CONTENTS

DATA MODELING - OLTP..... 3

 AUDIENCE 3

 INTRODUCTION 3

 PROJECT SCOPE 3

 DATABASE OBJECT NAMING CONVENTIONS 4

 GENERAL GUIDELINES 4

 NAMING CONVENTIONS 6

 PROGRAMMABLE OBJECTS NAMING CONVENTIONS 8

DATA MODELING – DATA MART 10

 AUDIENCE 10

 INTRODUCTION 10

 PROJECT SCOPE 10

 DATABASES 10

 GOOGLE BIG QUERY PROJECT NAMING CONVENTIONS 11

 OBJECT NAMING CONVENTIONS 13

 APPENDIX A – ATTRIBUTE SUFFIX 18

REVISION TABLE..... 19

DATA MODELING - OLTP

AUDIENCE

- Data modeling team
- Database developers
- Application developers involved in modeling

INTRODUCTION

As an initiative to designing the Loan Origination System (LOS), microservices, or when migrating data from system to system, the data modeling team set standards and general guidelines to model the system and maintain it. The standards will improve design, implementation, and support the new data models for the UWM LOS system.

PROJECT SCOPE

The standards are for database objects created and maintained for Loan Origination Systems and microservices. The standards are for the Online Transaction Processing system (OLTP) and do not apply to Data Warehouse.

	IMPORTANT! When updating, ensure that Destiny or Integra indicator columns have Booleans with a datatype of smallint instead of bit. Bit datatype breaks Destiny and will break production jobs.	
Schema	Using custom schemas is preferred, but dbo is also allowed. Authorization on a custom schema should be set to dbo. Create schemas based on logical groups of entities and if required, use vendor-specific schemas such as Cenlar, MRC (Mr. Cooper), etc.	
Tables and Entities	Include a clustered primary key to any table. - Choose a natural key for the primary key. - Include underscores for new tables if a database already has underscores for other tables in the database. - If there is no natural key, use a surrogate key as the primary key. - When tables are related, add foreign key constraints. - A prefix of USFS_ should be used on Integra tables, unless it's an extension of an Integra owned table, in which case use custom_. - If a table in Integra is greater than 100 columns, you must extend to a new table and put the _ext01 suffix. This must match the casing of the table you are extending. - For setup tables, you must have enumerated values, but you should not use Sequence or Identity. Manually set the values to ensure consistency across environments. Surrogate key values populate from a sequence number. - Identity keys are used only when it is necessary for users to protect the identity's column value. - Do not create max+1 OR identity keys. Include four standard columns for every entity: CreatedBy CreatedDateTime ModifiedBy ModifiedDateTime Give table and column names a singular functional name. When updating or changing an existing table and/or column in Integra, grandfather in existing in place Integra standards or patterns. IMPORTANT! When creating a new table, follow the Data Modeling Standards <i>unless</i> it will break code or cause issues in the Integra database tables. Exceptions <i>must</i> be approved by a Data Architect. NOTE: Omit spaces and underscores. NOTE: In Shore1 Integra Database, all <i>new</i> tables should include USFS_ as a prefix.	PK_TableName NOTE: For integra, you must use Dynace_Sequence function to get next sequence. FK_ChildTableName_ColumnName_ParentTableName NOTE: If you have a field populated by a sequence you should have a default set on surrogate key that calls the sequence. NOTE: CreatedBy; ModifiedBy should be populated from SYSTEM_USER. NOTE: CreatedBY ModifiedBy should be VARCHAR(128). Borrower, LoanCondition

NAMING CONVENTIONS

Name	Convention	Functional Example
Constraints and Indexes	The list below shows how to pattern the following keys: Check Constraints: CK_TableName_ColumnName - Start with the prefix CK and an underscore. - Add the table name and an underscore. - Add the column name. Default Constraints: DF_TableName_ColumnName - Start with the prefix DF and an underscore. - Add the table name and an underscore. - Add the column name. Foreign Keys: FK_ChildTableName_ColumnName_ParentTableName - Start with the prefix FK and an underscore. - Add the child table name and an underscore. - Add the abbreviated column name and an underscore. - Add the parent table name. Nonclustered Indexes: NCI_TableName_FunctionalName - Start with the prefix NCI and an underscore. - Add the table name and an underscore. - Add the functional name. Nonclustered Index with Include Column Names: NCI_TableName_ColumnName_INC - Start with the prefix NCI and an underscore. - Add the table name and an underscore. - Add the column name and an underscore. - Add INC. Filtered Indexes: FI_TableName_FunctionalName - Start with the prefix FI and an underscore. - Add the table name and an underscore. - Add the functional name. Filtered Index with Include Column Names: FI_TableName_ColumnName_INC - Start with the prefix FI and an underscore. - Add the table name and an underscore. - Add the column name and an underscore. - Add INC	CK_Borrower_Gender DF_Broker_ActiveIND FK_custom_loanmain_Idbid_Irid_loan_main NCI_Loan_Submission NCI_Loan_SubmissionDate_INC FI_ClosedLoans FI_ActiveBroker_INC

[illegible]

PROGRAMMABLE OBJECTS NAMING CONVENTIONS

Name	Convention	Functional Example
Programmable Objects	The list below shows how to pattern the following objects:	
	NOTE: It is necessary for all SQL statements to be a programmable object.	
	Indexed View:	
	NOTE: Indexed views must have an index.	
	ivw_FunctionalName - Start with the prefix ivw and an underscore. - Add the functional name.	ivw_CloserReviewQueue
	Procedure:	
	usp_select_FunctionalName Select will <i>fetch</i> data and only have one select statement. The functional name can be a table name.	usp_select_BorrowerDetail
	usp_insert_FunctionalName Setters <i>insert</i> data into multiple tables.	usp_insert_BorrowerDetail
	usp_create_FunctionalName - Create a record and then return the value of the record – this procedure can have an insert and select statement – - Start with the prefix usp and an underscore. - Add create and an underscore. - Add the functional name.	usp_create_MortgageRate
	usp_update_FunctionalName - Update data in multiple tables – - Start with the prefix usp and an underscore. - Add update and an underscore. - Add the functional name.	usp_update_BorrowerDetail
	usp_merge_FunctionalName - Insert merge statements only – - Start with the prefix usp and an underscore. - Add merge and an underscore. - Add the functional name.	usp_merge_PropertyDetail
	NOTE: Never use an actual merge statement. Only use update/insert.	
	usp_delete_FunctionalName - Delete data from multiple tables – - Start with the prefix usp and an underscore. - Add delete and an underscore. - Add the functional name.	usp_delete_BorrowerDetail
	When other use cases do not apply – usp_FunctionalName Start with the prefix usp and an underscore. Add the functional name.	usp_ClosedLoansNotFunded

Programmable Objects	Scalar Function: ufns_FunctionalName - Start with the prefix ufns and an underscore. - Add the functional name.	ufns_LoanStatus
	Sequence: seq_TableName_ColumnName - Start with the prefix seq and an underscore. - Add the table name and an underscore. - Add the column name.	seq_Loan_LoanID
	Table Value Function: ufnt_FunctionalName - Start with the prefix ufnt and an underscore. - Add the functional name.	ufnt_BorrowerDetail
	Table Types: utt_TableName utt_FunctionalName - Start with the prefix utt and an underscore. - Add the table name or functional name.	utt_Property utt_CustomLoanInformation
	Triggers: trg_TableName_FunctionalName - Start with the prefix trg and an underscore. - Add the table name and an underscore. - Add the functional name.	trg_Loan_LoanStatus
	View: uvw_FunctionalName - Start with the prefix uvw and an underscore. - Add the functional name.	uvw_NewConditionsReview

DATA MODELING – DATA MART

AUDIENCE

- Data services team
- Database developers

INTRODUCTION

The following are guidelines for developing SQL server data warehouse and datamart components used by the data services team. These are considered guidelines vs standards as there may be valid reasons for deviation. However, in the interest of maintainability and ease of code reviews these guidelines should be followed unless there is an overriding reason for deviation.

Use this document when creating your SQL server components and performing a code review. If a question arises whether a component should be modified based on a review the following should be considered:

- Will the code not perform as expected based on requirements? (i.e., result will be wrong, will not perform in a reasonable amount of time or will cause routine to error/fail).
- Does it conform to documented guidelines?

NOTE: Questions related to style should not be grounds for revisions in a code review.

APPLICABILITY:

Databases for SSRS reporting, SSIS and ETL components to be used by Data Services functions. Typically covers ShoreDStaging, ShoreDW, ShoreDM, DM, RPT_RVM, and other databases consumed applications supported by the Data Services team.

PROJECT SCOPE

These standards are for the Date Warehouse and Data Marts only. They do not apply to the Online Transaction Processing system (OLTP). Naming standards and correct data typing should be followed.

DATABASES

GENERAL INFORMATION

Database usage and patterns will be documented in a separate artifact, TBD.

Use existing standards and patterns when making minor changes to current data structures. These new standards apply primarily to new development, major refactoring of existing data warehouse apps and to existing structures which currently follow the standards.

GOOGLE BIG QUERY PROJECT NAMING CONVENTIONS

- **Project**
- **Data Set**
- **Object**

Name	Convention	Functional Example
Attributes	Google sorts by ASCII so lowercase and uppercase are considered different and will be nowhere near each other. PascalCase is enforced.	
Data Service Object	Raw Layer PR_{ENV}_{Layer}_{IngestionMethod}_{DB}_{Schema}_{Table} PR_INT_Raw PR_STG_Raw PR_PRD_Raw - Start with the prefix PR and an underscore. - Add the environment in all caps and an underscore. - Add the ingestion method name and an underscore. - Qlik - Talend - Kafka - Add the database name and a period. - Add the schema name and a period. - Add the object name as it is in source other than in pascal case. Prep Layer PR_{Env}_{Layer}_{Purpose}_{Desc}_{Type} PR_INT_Prp PR_STG_Prp PR_PRD_Prp - Start with the prefix PR and an underscore. - Add the environment and a period. - Add the purpose and a period. - Add the Description and an underscore. - Add UVW for Unmaterialized View or MVW for Materialized View. IMPORTANT! The delineation of tables, procedures, and more will be determined in the future. Serving Layer PR_{Env}_{Layer}_{Business}_{Desc}_{Type} PR_INT_Srv PR_STG_Srv PR_PRD_Srv - Start with the prefix PR and an underscore.	Raw Layer Example: PR_INT_Raw_Qlik_Integra.dbo.Loan_Main Prep Layer Example: PR_INT_Prp.Conform.Borrower_UVW Serving Layer Example: PR_INT_Srv.Sales.Fact_Description_UVW

	• Add the environment and a period. • Add the user or business unit and a period. • Add the prefix and an underscore. NOTE: The prefix is whatever the type is. In this case, Dim and Fact are a prefix not a postfix. • Add the Description and an underscore. • Add UVW for Unmaterialized View or MVW for Materialized View for the postfix. NOTE: Serving Layer for projects that are for specific use cases (the business units). Add the business unit into the name instead of the SRP and add the Grainlevel as the postfix.	Serving Layer Example with specific use case: PR_INT_Sales.Sales.Description_Grainlevel
--	--	---

OBJECT NAMING CONVENTIONS

Name	Convention	Example
Schema	Each table will have a schema that relates to the phase of the process: - src – schema is used for tables that store data from the source system(s) - stg – schema is used for tables that receive data from src schema tables - dm – schema is used for tables that receive data from stg schema tables - ctrl– schema is used for tables that receive data to manage the environment For existing schema, discussion may be required to change it to the new guidelines. Identify views and objects consumed by reports and other external applications by using 'interface'.	Cenlar MRC ShoreDW Dominate
Tables – Naming	Use Pascal case for tables, followed by _ (underscore) type. Core table name will capitalize the first letter of each word or abbreviation in the name. Followed by table type in lower case. TableName_type Types include, but are not limited to: Bridge: (Kimball standard hierarchy; sits between a fact table and a dimension table and is used to resolve a many-to-many relationship) Crosswalk: Mapping of natural keys from one or more source to corresponding surrogate keys. Supports conformed dimensions. Dimension: Base table which contains attributes used to filter facts. Fact: Contains metrics and measures related to a business process. History: Snapshot of information over a period of time. Similar to fact table but contains broader mix of measures and attributes. All tables should have a reference to a creation date at minimum, and modification date if applicable. With the exception of History tables, the dates represent when the record was created in the target, not the source database. The dates will depend on the table type. Examples: If records will only be created and never modified then ok to omit ModifiedDateTime History: ProdAsOfDate / ProdAsOfDatetime - represents the datetime of the source image. If applicable, it must be provided by the source. Not the same as CreatedDatetime which should be a separate column on the table.	UserAccount_map LoanOfficerLicense_dim Table_bridge Loan_xwalk AccountExecutive_dim Submission_fact Loan_history Default: CreatedDateTime ModifiedDateTime NOTE: Data type should be DATETIME2(3).

Tables – Naming	Fact: Will often have a date as part of the key. Choose a name which fits the activity being measured. Slowly Changing Dimension (SCD): Type 1: No history, overwrite the changed data. Use a CreatedDatetime and ModifiedDatetime Type 2: Row versioning. Entire previous row is replaced by new record, regardless of number of attributes being modified. StartDatetime: identifies when the change was received in the data mart. Represents the start date of the new record in the DM. May not necessarily be the effective date of the change in the source system. This could be provided by a separate column. EndDatetime: identifies when the previous record was replaced by a new one. Since it is datetime, the EndDatetime must be equivalent to the StartDateTime of the next record. Type 3: Column versioning. Record contains original value, current value and datetime modified. A current record indicator must be used to identify the latest record in a history or SCD type2 table. When used with a filtered index this can be an effective way of selecting the latest record instead of scanning through all of the datetime columns.	i.e. TransactionDate, MeasurementDate, OrderDate, etc.
Table – Constraints Table – Constraints	Primary Key All tables must have a primary key. Add a prefix to clarify the origin of the key: - If a surrogate key within the datamart being created/modified, prefix with PK, followed by _table name - If a foreign key is brought in from a source outside of the datamart being created/modified, prefix with source followed by _table name Foreign Key Enforce foreign key relationships within the DM\DW. - Start with the prefix FK and an underscore. - Add the child table name and an underscore. - Add the abbreviated column name and an underscore. - Add the parent table name. Nonclustered Index - Start with the prefix NCI and an underscore. - Add the table name and an underscore. - Add the functional name. Filtered Indexes: - Start with the prefix FI and underscore. - Add the table name and an underscore. - Add the functional name.	PK_borrower Destiny_property  dm_standards_diagram1.pdf FK_ChildTableName_ColumnName_ParentTableName NCI_TableName_FunctionalName FI_TableName_FunctionalName

	Filtered Index with Include Column Names: - Start with the prefix FI and an underscore. - Add the table name and an underscore. - Add the column name and an underscore. - Add INC. Check Constraints - Start with the prefix CK and an underscore. - Add the table name and an underscore. - Add the column name	FI_TableName_ColumnName_INC CK_TableName_ColumnName
View	View - Start with the prefix uvw and an underscore - Add the functional name Indexed View - Start with the prefix ivw and an underscore. - Add the functional name	uvw_FunctionalName ivw_FunctionalName
Columns	Use Pascal case for columns, followed by attribute suffix describing usage. Capitalize the first letter of each word or abbreviation in the name. NOTE: Refer to Appendix A for attribute suffix.	ServicerLoanSID NoteDate BaseMortgageAmt
Indicator and Flag Field	Define indicator fields as a data type BIT; values cannot be null. - Values must be 0 or 1. - Column must be constrained with a default value of 0. Define flag columns as a data type char(1); make the values nullable. - There are three possible values: - Y, N, NULL. - Define a check constraint on the column with the possible values. Setup tables require all the enumerated values. Include a row for UNKNOWN.	0=False, 1=True CHAR (1)
Stored Procedure - Select	usp_select_FunctionalName Select will fetch data and only have one select statement. The functional name can be a table name. Any stored procedure that retrieves data from a table(s). <schema>.usp_Get_<Purpose> Ok to use either convention.	usp_select_BorrowerDetail ctrl.usp_Get_LastCompleted Process
Stored Procedure - Insert	usp_insert_TableName - Insert data into one table - Start with the prefix usp and an underscore. - Add insert and an underscore. - Add the table name. Any stored procedure that inserts data into a table(s).	usp_insert_ZipCode src.usp_Load_SalesFact – load will be for loading tables, can be one or combination of insert, upd, del

	<schema>.usp_Load_<Table Name>	
Stored Procedure – Load Multiple tables	usp_insert_FunctionalName • Setters insert data into multiple tables	usp_insert_BorrowerDetail
Stored Procedure – Update	usp_update_FunctionalName • Update data in multiple tables ○ Start with the prefix usp and an underscore ○ Add update and an underscore ○ Add functional name	usp_update_BorrowerDetail
Stored Procedures - Delete	usp_delete_FunctionalName • Delete data from multiple tables • Start with the prefix usp and an underscore • Add delete and an underscore • Add Functional Name	Usp_delete_BorrowerDetail
Stored Procedure - Create	usp_create_FunctionalName Create a record then return the value of the record – this procedure can have an insert and select statement • Start with the prefix usp and an underscore • Add create and an underscore • Add Functional Name	usp_create_BorrowerDetail
Stored Procedure - Master	usp_master_FunctionalName Procedure that executes multiple calls to other sprocs, aka Driver. • Start with the prefix usp and an underscore • Add Master and an underscore • Add the functional name	usp_master_LoadDaily
Stored Procedure – Merge	usp_merge_FunctionalName Uses update/insert/delete statements • Start with the prefix usp and an underscore. • Add merge and an underscore. • Add the functional name.	usp_merge_LoanConditions

Function – Table Valued	ufnt_FunctionalName A table-valued function is a user-defined function that returns data of a table type. - Start with the prefix ufnt and an underscore - Add the table name or functional name	ufnt_CheckProcessOutcome
Function - Scalar	ufn_FunctionalName A scalar valued function is a function that takes zero or more values but returns a single value - Start with the prefix ufn and an underscore - Add the functional name	ufn_BorrowerDetail
Sequence Object	seq_TableName_ColumnName - Start with the prefix seq and an underscore. - Add the table name and an underscore. - Add the column name	seq_Loan_LoanID
Table Type Object	utt_TableName utt_FunctionalName - Start with the prefix utt and an underscore. - Add the table name or functional name	utt_Property utt_CustomLoanInformation
Trigger	trg_TableName_FunctionalName - Start with the prefix trg and an underscore. - Add the table name and an underscore. - Add the functional name	trg_Loan_LoanStatus

APPENDIX A – ATTRIBUTE SUFFIX

No underscores between name and suffix

Domain	Definition	Example	Data type/ Scale
Address	Line 1 and line 2 of an address	Address1, Address2	Must be VARCHAR(100)
City	City component of an address	City	Must be VARCHAR(100)
State code	US state code	State	Must be VARCHAR(2)
StateName	US State name	StateName	Must be VARCHAR(100)
Zipcode	US Postal ZipCode standard is 10-characters (12345-1234)	Zip	Must be VARCHAR(10)
County	County component of an address	County	Must be VARCHAR(100)
Code Identifier	Generally numeric identifier, typically associated with key/foreign key columns.	ID	Depends on usage
Short Description	String value of 20 characters or less	ShortDesc	Must be VARCHAR(20)
Long Description	String value > 20 characters	Desc	Must be > VARCHAR(20)
Date	Date without timestamp	Date	Must be DATE
Datetime	Date including timestamp	Datetime	Must be DATETIME
DatetimeUTC	Date including timestamp in UTC	DatetimeUTC	Must be DATETIMEOFFSET
Flag	Boolean value represented by character (ie, Y or N)	Flag	Must be CHAR(1)
Indicator	Boolean value represented by bit (ie, 1 or 0)	IND	Must be BIT (avoid using with Destiny app)
Amount	A monetary amount, related to currency. Does not include other domains such as weight, mass, volume.	Amt	Must be DECIMAL(13,2)
Code String	Character identifier, typically associated with key/foreign key columns	SID	Must be VARCHAR(20)
Name	Formal description used to identify an object. Usually a title or legal classifier.	Name	Must be VARCHAR(100)
Percent	Numeric value representing a percentage	Pct	Must be DECIMAL (N, 5) 100% should be stored as 100.0
Phone Number	Character value representing US phone number (including area code) without dashes.	Phone	Must be VARCHAR(10)

REVISION TABLE

Date Created	Author	Description
02/15/2023	M. Hirth, Tech Writer Specialist	Combined DataMart and Data Modeling Guides and updated standards
01/25/2024	M. Hirth, Tech Writer Specialist	Added Google Big Query Product Naming Conventions section.